EJERCICIO 1. (4 Puntos) **Modelado conceptual**

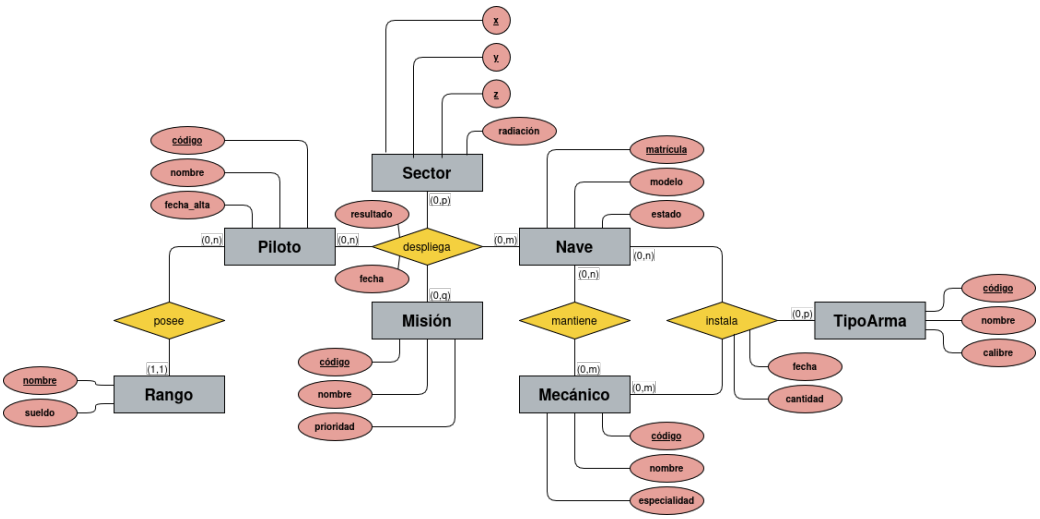
Queremos guardar información sobre las operaciones tácticas y logísticas de la flota colonial. Estas involucran a pilotos identificados por un código único, junto con su nombre y fecha de alistamiento. Todo piloto debe tener asociado obligatoriamente un único rango militar, el cual lleva asociado un sueldo base.

Las naves de combate, identificadas por su matrícula, y con modelo y estado actual, son mantenidas por mecánicos (con código único, nombre y especialidad). Son modelos antiguos, por lo que cualquier mecánico puede mantener cualquier tipo de nave. Además, se realizan tareas de artillado, en las que los mecánicos instalan diferentes tipos de arma (código único, nombre y calibre) en las naves que, dependiendo del tamaño, pueden acoplar más de una. Esta acción de instalación requiere registrar obligatoriamente quién y cuándo la realizó y la cantidad de munición instalada.

Finalmente, el CIC organiza el despliegue de las fuerzas. Un despliegue es una acción que vincula de forma simultánea a un piloto, la nave que pilota, la misión asignada (código único, nombre y prioridad) y el sector espacial de destino (identificado de forma única e indivisible por sus coordenadas x, y, z, registrando además su nivel de radiación). De cada despliegue se debe almacenar la fecha y el resultado del mismo, teniendo en cuenta que un mismo piloto, nave, misión o sector puede participar en múltiples despliegues distintos a lo largo del tiempo.

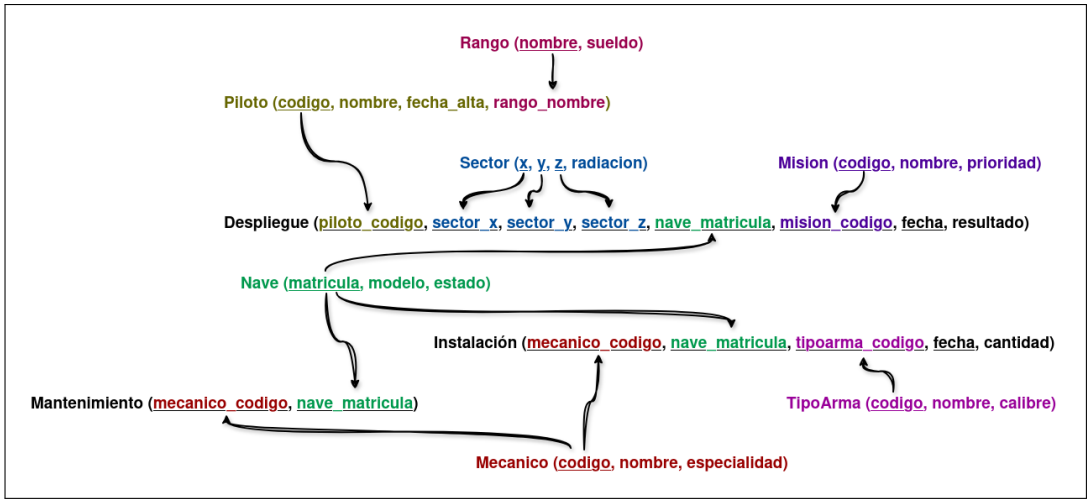
(a) (3 Puntos) Realiza un modelo conceptual de datos mediante la técnica del modelo entidad-relación de Chen que modele el sistema descrito.

Solución propuesta:



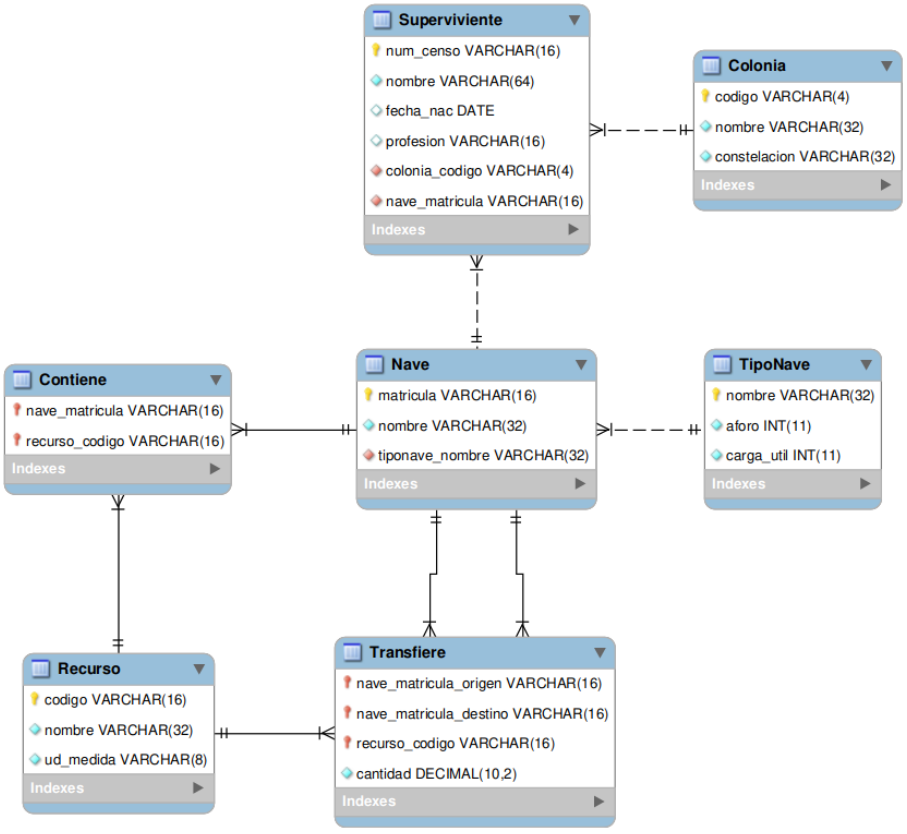
(b) (1 Punto) Realiza el paso a tablas del diagrama entidad relación propuesto.

Solución propuesta:



EJERCICIO 2. (5 Puntos) SQL

Disponemos de un análisis para el sistema de gestión logística y demográfica de la flota de supervivientes de las Doce Colonias. El diagrama E-R del análisis es el siguiente:



(a) (½ Punto) Obtener el nombre de las colonias que tienen más de 1000 supervivientes actualmente vivos en la flota.

Solución propuesta:

```sql
SELECT c.nombre
FROM Colonia c
  JOIN Superviviente s ON c.codigo = s.colonia_codigo
GROUP BY c.codigo, c.nombre
  HAVING COUNT(s.num_censo) > 1000;
```

(b) (½ Punto) Calcular la cantidad total transferida de cada recurso (mostrando el nombre del recurso) desde la nave origen `Scylla` hacia cualquier otra nave.

Solución propuesta:

```sql
SELECT r.nombre, SUM(t.cantidad) AS total_transferido
FROM Transfiere t
  JOIN Recurso r ON t.recurso_codigo = r.codigo
  JOIN Nave n ON t.nave_matricula_origen = n.matricula
WHERE n.nombre = 'Scylla'
GROUP BY r.codigo, r.nombre;
```

(c) (1 Punto) Para cada colonia, obtener el nombre del superviviente más anciano.

Solución propuesta:

```sql
SELECT c.nombre AS nombre_colonia, s.nombre AS
   superviviente_anciano
FROM Colonia c
  JOIN Superviviente s ON c.codigo = s.colonia_codigo
WHERE s.fecha_nac = (
  SELECT MIN(s2.fecha_nac)
  FROM Superviviente s2
  WHERE s2.colonia_codigo = s.colonia_codigo
);
```

(d) (1 Punto) Crea una función que, dada la matrícula de una nave, devuelva cuántas plazas libres quedan.

Solución propuesta:

```sql
CREATE FUNCTION plazas_libres(p_matricula VARCHAR(16))
RETURNS INT
BEGIN
    DECLARE aforo_max INT;
    DECLARE num_pasajeros INT;
    DECLARE libres INT;

    SELECT tn.aforo INTO aforo_max
    FROM Nave n
    JOIN TipoNave tn ON n.tiponave_nombre = tn.nombre
    WHERE n.matricula = p_matricula;
```

```
    SELECT COUNT(*) INTO num_pasajeros
    FROM Superviviente
    WHERE matricula_nave = p_matricula;

    SET libres = aforo_max - num_pasajeros;

    RETURN libres;
END;
```

(e) (1 Punto) Crea un *trigger* que, antes de asignar a un nuevo superviviente a una nave, compruebe si hay plazas libres. Si no las hay, debe lanzar un error y cancelar la inserción (se puede usar la función del ejercicio anterior, puedes usarla en este ejercicio).

Solución propuesta:

```
-- Usamos la función anterior y así no repetimos código
CREATE TRIGGER trg_control_aforo
BEFORE INSERT ON Superviviente
FOR EACH ROW
BEGIN
    DECLARE plazas INT;

    SET plazas = plazas_libres(NEW.nave_matricula);

    IF plazas <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: A tope de aforo';
    END IF;
END;
```

(f) (1 Punto) Crea un procedimiento que registre una transferencia de recursos. Debe recibir la matrícula origen, matrícula destino, código del recurso y cantidad. Debe verificar que la nave origen y destino no sean la misma y que la nave destino cuenta con suficiente capacidad para almacenar los recursos.

Solución propuesta:

```
CREATE PROCEDURE sp_registrar_transferencia(
    IN matricula_origen VARCHAR(50),
    IN matricula_destino VARCHAR(50),
    IN codigo_recurso VARCHAR(50),
    IN cantidad DECIMAL(10,2)
)
BEGIN
    DECLARE carga_max DECIMAL(10,2);
    DECLARE carga_actual DECIMAL(10,2);

    IF matricula_origen = matricula_destino THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Mismo origen y destino';
    END IF;

    SELECT tn.carga_util INTO carga_max
    FROM Nave n
    JOIN TipoNave tn ON n.tiponave_nombre = tn.nombre_tipo
    WHERE n.matricula = nave_matricula_destino;

    SELECT SUM(cantidad) INTO carga_actual
    FROM Transfiere
    WHERE nave_matricula_destino = matricula_destino;

    IF carga_actual IS NULL THEN
        carga_actual := 0;
    END IF;

    -- No lo habéis visto, pero estas seis últimas líneas las
    -- podéis simplificar con COALESCE:
    --
    -- SELECT COALESCE(SUM(cantidad), 0) INTO carga_actual
    -- FROM Transfiere
    -- WHERE nave_matricula_destino = matricula_destino;
    --
    -- O con CASE, que es al final en lo que se traduce el
    -- COALESCE por debajo:
    --
    -- SELECT CASE
    --   WHEN SUM(cantidad) IS NULL THEN 0
    --   ELSE SUM(cantidad)
    -- END INTO carga_actual
    -- FROM Transfiere
    -- WHERE nave_matricula_destino = matricula_destino;


    IF (carga_actual + cantidad) > carga_max THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Sin capacidad de carga';
    ELSE
        INSERT INTO Transfiere (nave_matricula_origen,
            nave_matricula_destino, recurso_código, cantidad)
        VALUES (matricula_origen, matricula_destino,
            codigo_recurso, cantidad);
    END IF;
END;
```

EJERCICIO 3. (1 Punto) **Programación contra bases de datos.**

Completa los huecos (marcados con `TODO`) del siguiente código fuente en Python de tal manera que se satisfaga la funcionalidad indicada en los *docstrings*.

```
import mysql.connector

# Conexión al mainframe del CIC de la Galactica
conn = mysql.connector.connect(
```

```
    host="bs.local",
    user="adama",
    password="SoSayWe4ll",
    database="flota",
)
cursor = conn.cursor()

def obtener_pasajeros_nave(matricula):
    """Obtiene los supervivientes asignados a una nave en concreto.

    :param matricula: La matrícula identificativa de la nave.
    :return: Lista de nombre y profesión de sus supervivientes.
    """
    # TODO
```

Solución propuesta:

```
    consulta = """
    SELECT nombre, profesion
    FROM Superviviente
    WHERE matricula_nave = %s;
    """
```

```
    cursor.execute(consulta, (matricula,))
    return cursor.fetchall()

def registrar_transferencia(origen, destino, recurso, cantidad):
    """Registra un movimiento de recursos entre dos naves distintas.

    :param origen: Matrícula de la nave que cede el recurso.
    :param destino: Matrícula de la nave que recibe el recurso.
    :param recurso: Código del recurso a transferir.
    :param cantidad: Cantidad numérica a transferir.
    :return: True si se ha registrado el movimiento o False si no.
    """
    # TODO
```

Solución propuesta:

```
    if origen != destino:
        cursor.execute(
            """INSERT INTO Transfiere (nave_matricula_origen,
               nave_matricula_destino, recurso, cantidad)
               VALUES (%s, %s, %s, %s)""",
            (origen, destino, recurso, cantidad)
        )
        conn.commit()
        return True
    return False
```

```
def naves_sin_supervivientes():
    """Localiza naves automatizada sin supervivientes asignados.

    :return: Lista con las matrículas de estas naves.
    """
    # TODO
```

Solución propuesta:

```
    consulta = """
    SELECT matricula
    FROM Nave n
    WHERE NOT EXISTS (
        SELECT 1
        FROM Superviviente s
        WHERE s.nave_matricula = n.matricula
    );
    """

    cursor.execute(consulta, (matricula,))
    return cursor.fetchall()

conn.close()
```